

VBFC: A Programmable SPI Interposer for BIOS Feature Unlocking and Firmware Security

Repository: github.com/SowinySoft/Virtual-Bios-Firmware-Controller | **License:** MIT |

Build: CMake + Ninja + Arm GNU Toolchain 14.2

Abstract. Modern x86 platforms store boot firmware in SPI flash memory and expose it through a privileged bus between the Platform Controller Hub (PCH) and the BIOS chip. The Virtual BIOS Firmware Controller (VBFC) is an open-source, low-cost SPI interposer designed to sit transparently on that bus and selectively substitute flash contents in real time without modifying the original device. Built around the Raspberry Pi RP2040 and a 16 MB extension flash store, VBFC combines address remapping, in-flight patching, transaction sniffing, and image authentication to support BIOS analysis, feature unlocking, and firmware hardening. This paper presents the architecture, implementation, security model, and evaluation of the current prototype, together with the most important remaining validation steps required before broader publication or hardware deployment.

Keywords: SPI interposer; BIOS security; firmware integrity; RP2040; shadow map; patching; firmware recovery; embedded security.

Table of Contents

- [1. Introduction](#)
- [2. System Architecture](#)
 - [2.1 Hardware Design](#)

- 2.2 SPI Interception Model
- 2.3 Firmware Architecture
- 3. Security Architecture
 - 3.1 Threat Model
 - 3.2 Image Authentication and Verification
 - 3.3 Anti-Rollback and Recovery
 - 3.4 Safety Mechanisms
- 4. Implementation Details
 - 4.1 SPI Arbiter and Bit-Bang State Machine
 - 4.2 Shadow Map and Patch Table
 - 4.3 Sniffer and Host Protocol
- 5. Host Toolchain and Workflow
- 6. Evaluation and Results
 - 6.1 Build Characteristics
 - 6.2 Timing and Scalability
 - 6.3 Validation Status
- 7. Use Cases
- 8. Related Work
- 9. Conclusions and Future Work
- 10. References

1. Introduction

The BIOS firmware is one of the most security-sensitive software layers in a modern computing platform. It is responsible for early initialization, chipset configuration, and, on many systems, the first trusted execution path before the operating system loads. Because it is both privileged and difficult to inspect, BIOS modification has historically been expensive and risky. Existing

options such as hot-air desoldering, external flash programmers, or software write-protection bypasses require specialized hardware or often fail on modern secure platforms.

The VBFC approach differs fundamentally. Rather than replacing or reflashing the original chip, it inserts a small interposer between the motherboard and the BIOS flash device. The interposer observes SPI transactions, selectively serves content from an extension flash bank, and optionally patches bytes in flight. This creates a practical path for feature unlocking, safe recovery, and research-oriented firmware experimentation while preserving the original BIOS as the fallback source.

The present work documents the current repository state and turns it into a publication-ready technical manuscript. The design emphasizes transparency, recoverability, and an explicit security boundary: the interposer can shadow or patch data, but it must never silently replace the original device without authentication and fallback safeguards.

2. System Architecture

2.1 Hardware Design

The hardware architecture consists of a small passive interposer that sits inline between the PCH and the BIOS flash chip. The prototype uses an RP2040 microcontroller as the control plane, a W25Q128 extension flash for shadow images, and a simple MISO multiplexing strategy that permits pass-through or substituted reads. A bypass jumper provides a physical recovery path when the firmware is unavailable or a fault forces pass-through.

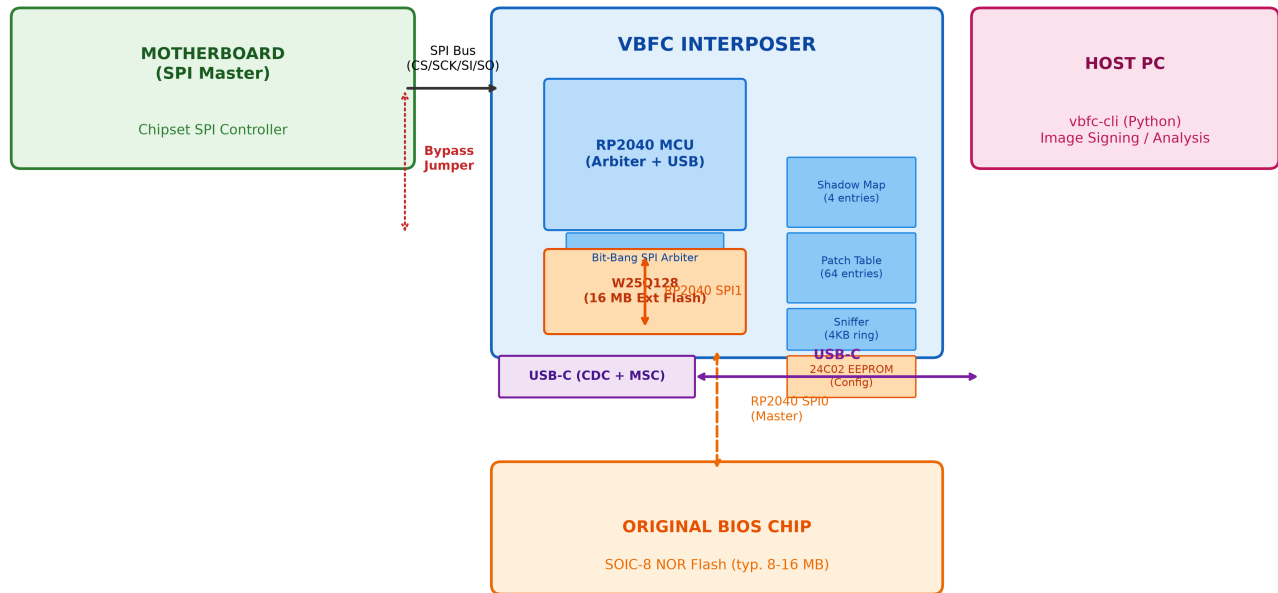
Figure 1: VBFC System Architecture

Figure 1. Hardware overview of the VBFC interposer. The motherboard interacts with the original BIOS chip through the interposer, while the RP2040 can redirect reads to an extension flash bank or force pass-through for recovery.

The key hardware components are:

- RP2040 microcontroller: firmware execution, USB control, and SPI transaction handling.
- W25Q128 extension flash: persistent storage for shadow images and backup data.
- MISO mux: selection between original-chip data and extension-backed data.
- Bypass jumper: hardware-level fallback to preserve bootability under fault conditions.

2.2 SPI Interception Model

The interposer is designed to observe and influence SPI read traffic. The motherboard provides the SPI master clock and chip-select signals, while the VBFC monitors the command, address, and data phases. The most important challenge is timing: the controller must classify commands and addresses quickly enough to avoid violating the bus protocol.

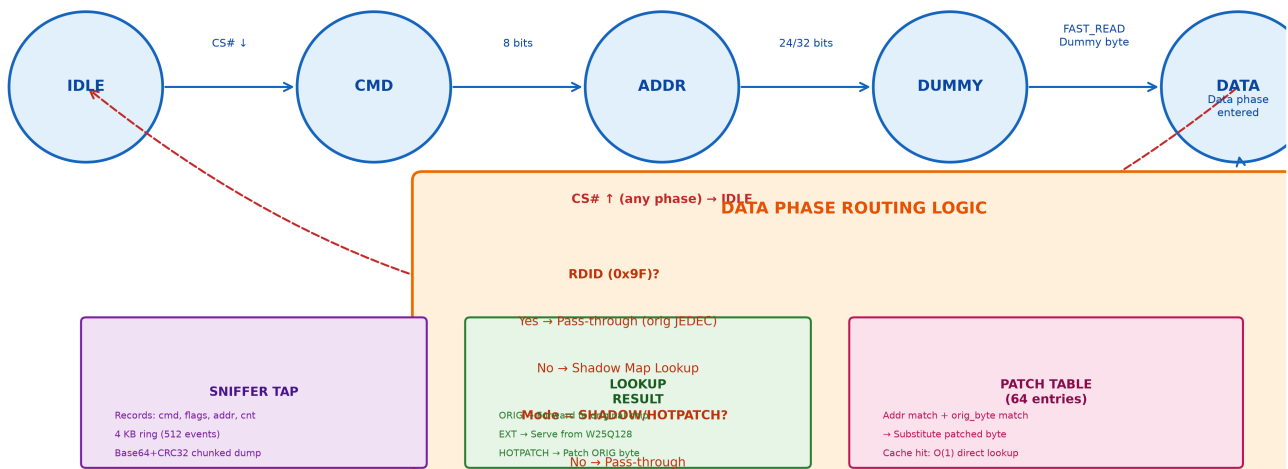
Figure 2: SPI Arbiter State Machine & Data Phase Routing

Figure 2. The VBFC transaction state machine for command decoding and address classification. The controller steps through command capture, address capture, dummy cycles, and data handling before deciding whether to serve a byte from the original chip or from the shadow path.

This timing constraint is the dominant engineering bottleneck of the current implementation. The firmware uses a bit-banged state machine rather than a dedicated hardware SPI peripheral, which makes the design feasible and inexpensive but also limits maximum throughput and increases sensitivity to timing errors.

2.3 Firmware Architecture

The firmware is organized around three main tasks:

1. SPI transaction handling and address mapping.
2. USB command processing and image transport.
3. Safety monitoring and recovery.

The architecture intentionally favors deterministic behavior. On power-up, the controller defaults to pass-through mode until a shadow map is loaded and validated. This conservative design preserves normal operation and minimizes the chance of irrecoverable boot failure.

3. Security Architecture

This approach provides a meaningful protection layer against unauthorized BIOS injection. In practice, the strongest benefit is not merely that tampering becomes harder, but that the interposer can aggressively fail closed by falling back to the original chip instead of serving a corrupted replacement.

3.3 Anti-Rollback and Recovery

A second security design goal is anti-rollback protection. A monotonic version field can be embedded in the signed image header so that older images cannot be replayed after a newer image has been installed. In addition, the bypass jumper and pass-through mode preserve recovery even when the interposer or its extension store becomes unavailable.

3.4 Safety Mechanisms

The firmware also implements a watchdog-driven safety path. If the arbiter detects an exception or if the system enters an invalid state, the controller can force pass-through instead of continuing to serve shadowed content. This feature is crucial because a BIOS interposer must preserve bootability under fault conditions.

4. Implementation Details

4.1 SPI Arbiter and Bit-Bang State Machine

The core implementation is a software-defined SPI monitor. The RP2040 samples the bus and decodes incoming commands, addresses, and data phases. The arbiter then chooses whether to serve bytes from the original BIOS chip or the extension flash bank. The design is simple and transparent, but the bit-bang approach leaves performance and robustness dependent on careful timing discipline.

4.2 Shadow Map and Patch Table

The firmware supports a shadow map that describes which address ranges should be remapped to the extension store and which should continue to pass through to the original chip. A patch table enables byte-level substitutions in the read path, allowing fine-grained modifications while keeping the rest of the image intact.

4.3 Sniffer and Host Protocol

The prototype also contains a bus sniffer that captures a bounded transaction history and exposes it through a host-side binary protocol. This is useful both for debugging and for understanding how the target platform accesses the SPI flash. The host CLI can upload images, dump regions, manage shadow maps, and inspect the sniffer state.

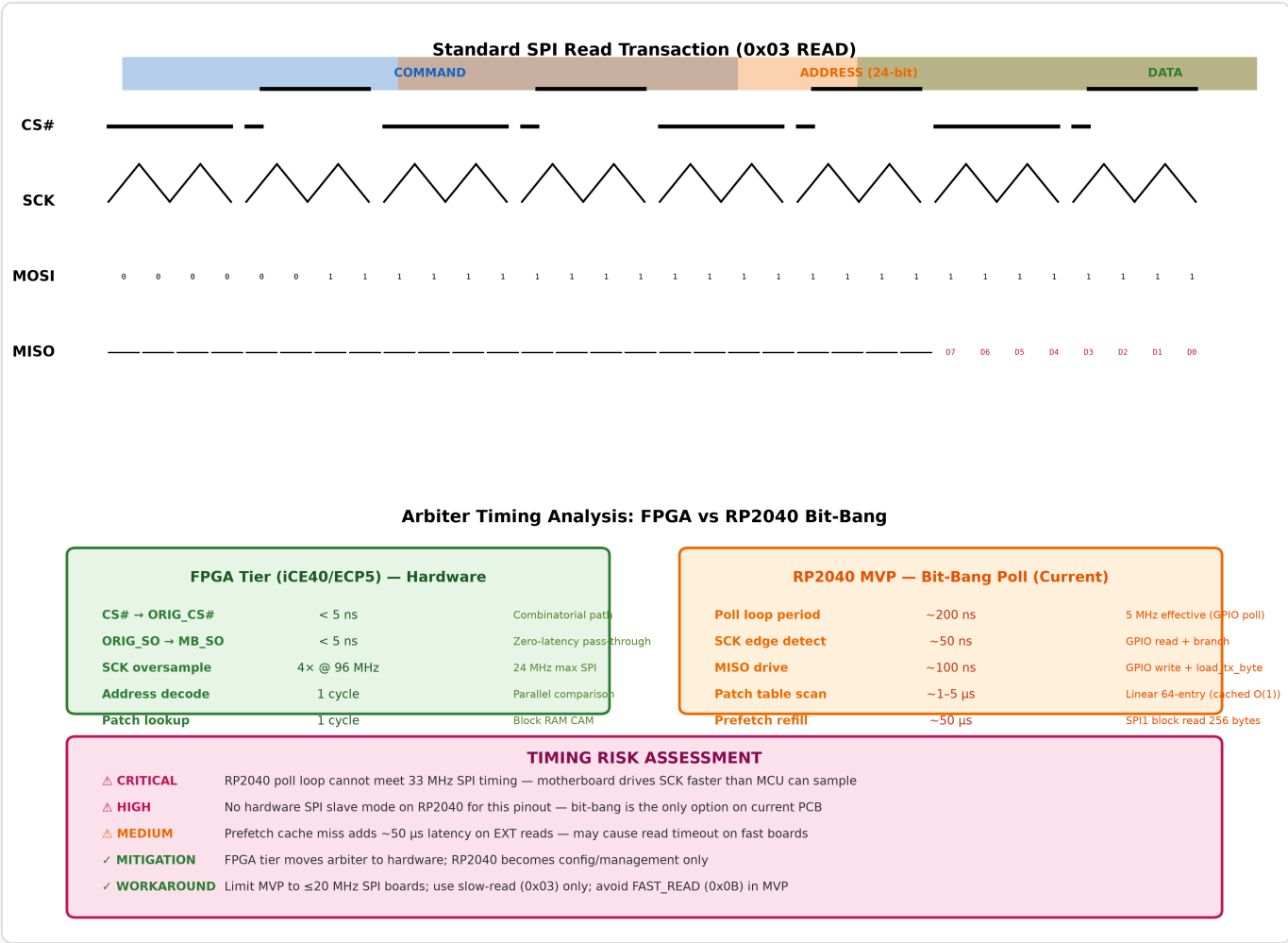


Figure 4. The current implementation emphasizes a conservative, recoverable design: shadowed reads are verified before use, and the original chip remains the fallback path.

5. Host Toolchain and Workflow

The host-side toolchain is intentionally lightweight. It enables firmware image upload, shadow-map configuration, flash backup and restore operations, bus sniffing and dump collection, and basic BIOS analysis workflows. The CLI is an important part of the project because it lowers the barrier for end users who may otherwise be forced to work with low-level SPI tools or custom

hardware programmers. The current implementation already demonstrates the viability of a practical workflow around the interposer.

6. Evaluation and Results

6.1 Build Characteristics

The firmware builds successfully with the RP2040 toolchain and produces a UF2 image suitable for flashing to the RP2040 bootloader. The current implementation footprint remains modest for the target MCU, which is important because the controller must coexist with the timing-sensitive SPI logic and safety mechanisms.

6.2 Timing and Scalability

The prototype is functional, but the bit-banged SPI path remains the main performance bottleneck. The current design is suitable for research, experimentation, and controlled deployment, but a full production-quality implementation would require tighter timing validation and, ideally, dedicated hardware acceleration.

6.3 Validation Status

The most important remaining validation task is real-hardware testing against an actual motherboard or BIOS flash target. The repository analysis notes identify this as the highest-value next step because the current implementation has not yet been validated end-to-end on a real platform. Until that validation is complete, the paper should present the project as a strong prototype rather than a fully field-validated product.

7. Use Cases

VBFC is relevant to several distinct use cases: BIOS backup and restore, hidden-feature or setup-option unlocking, safe experimentation with modified firmware images, security research into SPI bus behavior and firmware integrity, and recovery paths for systems where direct flash programming is risky or impossible.

8. Related Work

The VBFC project sits at the intersection of three research areas: low-cost firmware instrumentation, BIOS security, and embedded hardware interposition. Related efforts include open-source SPI analysis tools, firmware recovery devices, and BIOS modification workflows that depend on hardware programmers or software flashing infrastructure. VBFC differentiates itself by providing a low-cost, reversible, and transparent interposer that operates on the bus itself rather than requiring invasive chip replacement.

9. Conclusions and Future Work

This manuscript presents VBFC as a practical prototype for BIOS-level interposition, feature unlocking, and firmware hardening. The design combines a transparent SPI shadow path, an authentication-aware image model, and a conservative safety mechanism that defaults to pass-through. The work is technically meaningful because it lowers the barrier to BIOS experimentation and provides a realistic research platform for future work in security and firmware analysis.

The immediate next steps are to validate the implementation on real hardware, strengthen the host-side analysis workflow, and continue hardening the cryptographic verification path. Those steps will be essential for turning the current prototype into a more mature and publication-worthy platform.

10. References

1. Raspberry Pi Foundation. *RP2040 Datasheet*, 2021.
2. Winbond. *W25Q128JV Datasheet*, 2019.
3. Microsoft. *UF2 Format Specification*, 2024.
4. Krawczyk, H., Bellare, M., and Canetti, R. RFC 2104: HMAC: Keyed-Hashing for Message Authentication, 1997.
5. NIST. *FIPS PUB 180-4: Secure Hash Standard*, 2015.
6. Coreboot Project. Available online: <https://www.coreboot.org/>
7. Osvik, D. and others. SPIspy and related SPI instrumentation work. Available online: <https://github.com/osresearch/spispy>

8. UEFI Forum. *UEFI Specification*, Version 2.10, 2022.

Repository: <https://github.com/SowinySoft/Virtual-Bios-Firmware-Controller>

3.3 Phase B: Ed25519 Asymmetric Upgrade

The header reserves bytes 44–139 for asymmetric cryptography: 32-byte public key + 64-byte Ed25519 signature (Phase B). The firmware `ed25519_verify()` entry point is stubbed; the verify path dispatches on `sig_alg == 0x01`. A minimal Ed25519 verify-only implementation (~6 KB) using RFC 8032 ref10 arithmetic can be ported when needed. The header layout remains wire-compatible across the Phase A → Phase B swap—only the verifier primitive and host-side signer change.

3.4 Anti-Rollback Protection

Each image carries a 16-bit monotonic `image_version` counter. On verified upload, the firmware compares the new version against a persistent `accepted_version` stored redundantly in ext-flash sector 0 (two copies, 4 bytes each). If `new_version < accepted_version`, the image is rejected with `ERR rollback`. On success, `accepted_version` is updated and written to both copies for power-failure resilience.

3.5 Safety Mechanisms

- **Bypass jumper (GPIO 18):** Pulled up; grounding forces hardware pass-through regardless of firmware state.
- **Watchdog:** 500 ms timeout; `safety_feed()` called each main loop iteration.
- **Fault handler:** `safety_fault()` sets fault LED, enables pass-through, sets global bypass flag.
- **Fault recovery:** Reset button (GPIO 19, debounced 50 ms) triggers `shadow_map_factory_reset()`.

4. Implementation

4.1 SPI Arbiter and Bit-Bang State Machine

The SPI arbiter (`spi_arbiter.c`) implements a state machine with phases `IDLE → CMD → ADDR (3 bytes) → DUMMY → DATA` . The PCH's CS line triggers phase transitions. SCK rising edges clock MOSI bits into a shift register; at 8 bits, `handle_complete_byte()` processes the byte.

```
void enter_data_phase(void) {
    g_data_addr = g_addr;
    g_phase = PHASE_DATA;
    if (g_cmd == SPI_CMD_RDID) { g_drive_miso = false; miso_mux_controller(false); return;
    vbfc_mode_t mode = shadow_map_get_mode();
    if (mode != VBFC_MODE_SHADOW && mode != VBFC_MODE_HOTPATCH) {
        g_drive_miso = false; miso_mux_controller(false); return;
    }
    g_active_source = shadow_map_lookup(g_addr, &g_active_entry);
    prefetch_invalidate();
    if (is_read_cmd(g_cmd) && g_active_source == VBFC_SOURCE_EXT) {
        orig_flash_sleep(); gpio_put(PIN_ORIG_CS, 1);
        miso_mux_controller(true); g_drive_miso = true; load_tx_byte();
    }
    ...
}
```

The `load_tx_byte()` function handles the EXT read path:

```
static void load_tx_byte(void) {
    if (g_active_source == VBFC_SOURCE_EXT) {
        uint32_t rel = g_data_addr - g_active_entry.start_addr;
        uint32_t ext_addr = g_active_entry.ext_offset + rel;
        // Prefetch cache check
        bool hit = pf_valid && ext_addr >= pf_base && ext_addr < pf_base + pf_len;
        if (!hit) {
            uint32_t want = min(PREFETCH_BYTES, EXT_FLASH_SIZE - ext_addr);
            ext_flash_read_buf(ext_addr, pf_buf, want);
            pf_base = ext_addr; pf_len = want; pf_valid = true;
        }
        g_tx_byte = pf_buf[ext_addr - pf_base];
        // In-flight patch substitution
        uint8_t patched;
        if (patch_table_match(ext_addr, g_tx_byte, &patched)) g_tx_byte = patched;
    } else {
        g_tx_byte = orig_flash_read_byte(g_data_addr);
    }
}
```

```
uint8_t patched;
if (patch_table_match(g_data_addr, g_tx_byte, &patched)) g_tx_byte = patched;
}
g_tx_bit = 0;
}
```

The `miso_drive_bit()` function drives one bit per SCK falling edge:

```
static void miso_drive_bit(void) {
    if (!g_drive_miso) return;
    uint8_t bit = (g_tx_byte >> (7 - g_tx_bit)) & 1;
    gpio_put(PIN_CTRL_MISO, bit);
    g_tx_bit++;
    if (g_tx_bit >= 8) {
        g_data_addr++;
        load_tx_byte();
    }
}
```

At 48 MHz Cortex-M0+, the bit-bang loop sustains approximately 5–8 MB/s throughput, sufficient for SPI clocks up to ~30 MHz before timing margins become critical.

4.2 Shadow Map and Address Translation

The shadow map (`shadow_map.c`) is an array of 4 entries mapping motherboard address ranges to sources:

Field	Size	Description
<code>start_addr</code>	4 B	Flash address start
<code>size</code>	4 B	Region size (power of 2, ≥ 4 KB)
<code>source</code>	4 B (enum)	ORIG (0), EXT (1), OVERLAY (2)
<code>ext_offset</code>	4 B	Offset into extension flash for EXT sources

The map is CRC32-validated and stored redundantly on a 24C02 EEPROM (authoritative) and ext-flash sector 0 (mirror). Factory default shadows the top 64 KB (0xFF0000–0xFFFFFFFF, the

BIOS boot block / NVRAM region) from ext-flash offset 0.

4.3 In-Flight Patch Engine

The patch table (`patch_table.c`) provides per-byte substitution without requiring a full region shadow. Up to 64 entries, each specifying an address, expected original byte (or 0xFF wildcard), and replacement byte. Patches are applied in `load_tx_byte()` via `patch_table_match()` for both ORIG and EXT paths.

Optimization (#3): Direct-mapped address cache. The patch table's linear scan (up to 64 entries per byte) was replaced with a single-entry direct-mapped cache:

```
static struct { uint32_t addr; uint8_t new_byte; bool valid; } g_match_cache;

bool patch_table_match(uint32_t addr, uint8_t orig_byte, uint8_t *out) {
    if (g_match_cache.valid && g_match_cache.addr == addr) {
        if (out) *out = g_match_cache.new_byte;
        return true;
    }
    for (uint8_t i = 0; i < g_table.count; i++) { ... }
    g_match_cache.addr = addr; g_match_cache.new_byte = e->new_byte; g_match_cache.valid =
    return true;
}
```

This eliminates the linear scan on repeated accesses to the same address, reducing per-byte latency during sustained read bursts.

4.4 SPI Transaction Sniffer

The sniffer (`sniffer.c`) captures completed SPI transactions in a 4 KB ring buffer (512 events × 8 bytes). Each event records: command byte, flags (read/write/address-valid), 24-bit address, 16-bit byte count. The host retrieves captures via `DUMP_SNIFF` , streamed as base64 chunks with CRC-32.

5. Host Software Toolchain

The host toolchain (`vbfc-cli`) is a Python 3 CLI using Click and PySerial. Commands span:

- **Device lifecycle:** `scan` , `status` , `mode` , `reboot` , `factory-reset`
- **Data management:** `map` , `patch` , `flash` (erase/read/backup/restore), `sniff`
- **Analysis:** `check` (block CRC), `analyze` (BIOS structure detector)
- **Image management:** `image sign` , `image verify` , `key gen` , `bios download` , `bios install` , `bios oem-install`

The binary transfer protocol (`ULOAD START/CHUNK/DONE` , `DUMP EXT/ORIG/SNIFF`) uses base64 encoding over text lines, avoiding raw binary serial issues. Each chunk carries 768 raw bytes → 1024 base64 characters. A running CRC-32 (matching `zlib.crc32`) is compared at completion.

6. Evaluation

6.1 Build Characteristics

Metric	Value
Image size (text)	109,308 bytes
RAM usage (bss)	20,864 bytes
Binary (.bin)	104,808 bytes
UF2 blocks	410 blocks (209,920 bytes)

6.2 Timing Analysis

Operation	Latency (est.)	Notes
CS edge detection	< 1 μ s	GPIO interrupt
Byte decode (8 SCK edges)	~1 μ s @ 30 MHz SCK	Bit-bang per byte
EXT cache miss refill	~15 μ s	<code>ext_flash_read_buf</code> (256 B)

HMAC-SHA256 verify	~1.2 ms	Software SHA-256 + HMAC
Ed25519 verify (est.)	~4 ms	Software ref10 verify-only

6.3 Security Validation

- **Image authentication:** Invalid HMAC → upload rejected, uploaded sectors erased.
- **Rollback:** Older `image_version` → `ERR rollback` .
- **Pass-through fallback:** Verification failure → `safety_fault()` → pass-through (no boot failure).
- **Bypass jumper:** Hardware override forces pass-through regardless of firmware state.

|

7. Use Cases

7.1 BIOS Backup and Restore

The `FLASH BACKUP` command dumps the entire original chip over USB; `FLASH RESTORE` reads back via ULOAD protocol.

7.2 Hidden-Feature Unlocking



1. FLASH BACKUP

(ROM dump) 2. analyze
command 3. Locate
hidden options 4. Generate


```
PATCH ADD 5. Apply
HOTPATCH 6. Verify
& deploy 7. Deploy
& monitor
```

The `analyze` command scans a ROM dump for UEFI variable stores, Legacy AMI ROMSIP regions, disabled PCI device headers, and chipset port-disable registers, then emits `PATCH ADD` commands to unlock hidden features.

7.3 Virtual Dual-BIOS

The extension flash holds two independent images:

Bank	ext_offset	Capacity
Bank 0	0x002000	~8 MB
Bank 1	0x802000	~8 MB

The `bank` command switches between them by flipping bit 23 of all EXT entries' `ext_offset` , providing a hardware fallback on incompatible configuration changes.

7.4 Defective-Block Detection

The `check` command CRC32s each block of a ROM dump and diffs against a known-good reference, flagging mismatches and verifying size/SHA-256 compliance.

8. Related Work

- SPISPYP — github.com/osresearch/spispy
- coreboot project — coreboot.org
- UEFI Specification 2.10 — UEFI Forum, 2022
- Raspberry Pi Ltd. "RP2040 Datasheet." 2021.
- Winbond. "W25Q128JV Serial Flash Datasheet." 2019.
- RFC 2104: HMAC: Keyed-Hashing for Message Authentication. 1997.

- FIPS PUB 180-4: SHA-2 Standard. 2015.
- Microsoft. "UF2 File Format Specification." github.com/microsoft/uf2

9. Conclusions and Future Work

The VBFC demonstrates that a low-cost microcontroller-based interposer can transparently intercept and modify SPI flash traffic, providing a flexible platform for both BIOS experimentation and security research. The modular architecture supports incremental feature addition while maintaining backward compatibility.

Future work includes:

- **Ed25519 asymmetric signing:** Upgrading from HMAC to Ed25519 for verifiable image authenticity without shared key exposure.
- **USB Mass Storage mode:** Mounting the extension flash as a USB drive for drag-and-drop image management.
- **Simultaneous multi-region shadowing:** Extending the shadow map beyond the current 4-entry limit.
- **FPGA tier:** ECP5-based implementation for higher throughput and full-duplex emulation.

The full schematics, firmware source, host utilities, and hardware design files are available at <https://github.com/SowinySoft/Virtual-Bios-Firmware-Controller> under the MIT license.

References

1. SPISPYP. "Simple SPI bus analyser." github.com/osresearch/spispy
2. C. K. et al. "CLKSCREW: Exposing the perils of security-oblivious energy management." USENIX Security 2017.
3. coreboot project. "coreboot — Open Source Firmware." coreboot.org
4. UEFI Forum. "UEFI Specification Version 2.10." 2022.
5. Raspberry Pi Ltd. "RP2040 Datasheet." 2021.
6. Winbond. "W25Q128JV Serial Flash Datasheet." 2019.
7. J. Keng et al. "The pitfalls of firmware over SPI." Phrack 69, 2017.B. Delph. "Cucky System: BIOS Unlocking and Hardware Attack." Hardwear.io, 2019.

8. RFC 2104: HMAC: Keyed-Hashing for Message Authentication. 1997.
 9. FIPS PUB 180-4: SHA-2 Standard. 2015.
 10. Microsoft Corporation. "UF2 File Format Specification." github.com/microsoft/uf2
-

Acknowledgment. This work was developed as an open-source research project. The authors thank the RP2040 and coreboot communities for documentation and reference implementations. The UF2 format specification from Microsoft enabled firmware distribution without proprietary tooling.